Gomolemo Dizamuhupe

PROG6212

ST10491441

GROUP 4

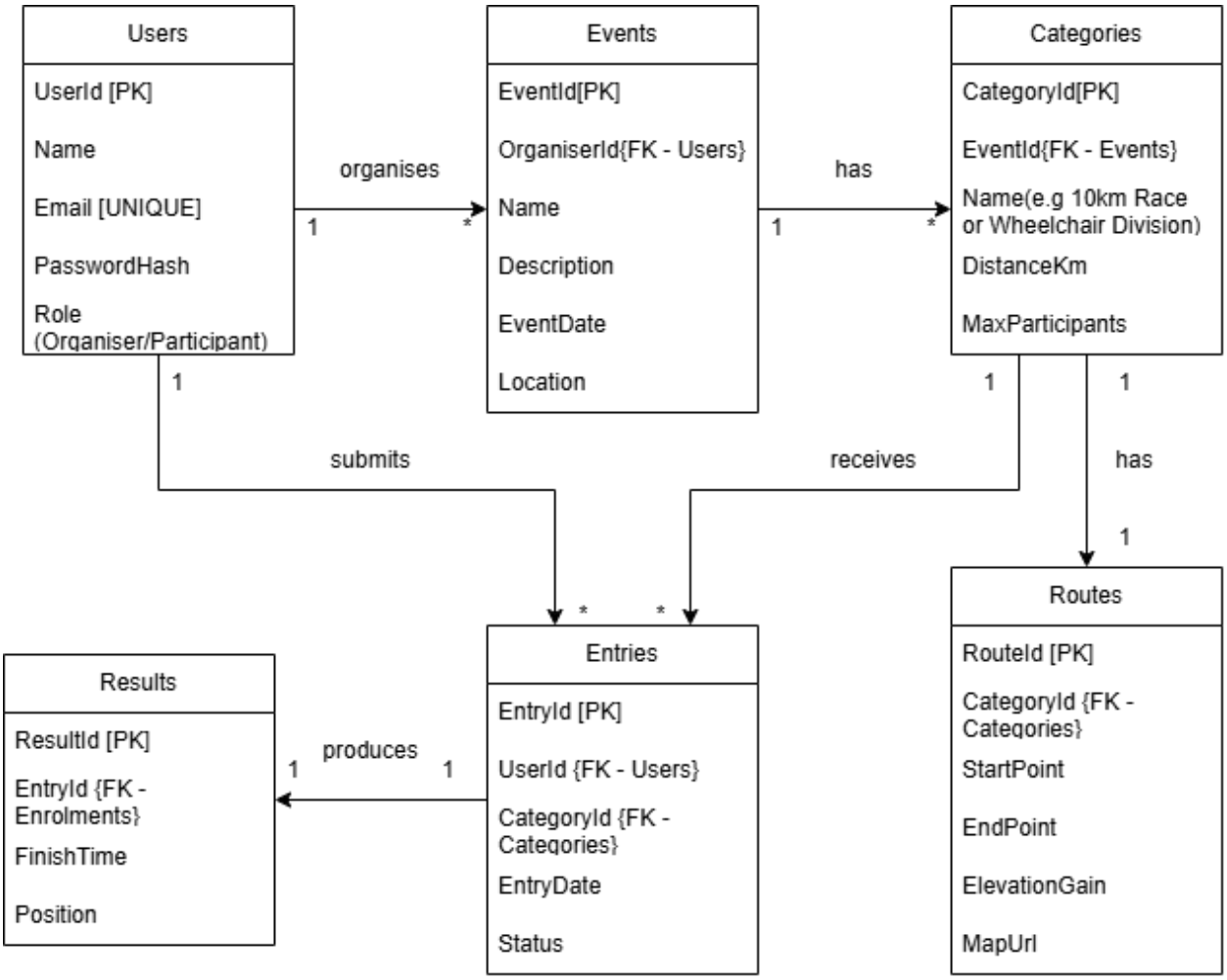
LINKS

1. YOUTUBE: PROG6212 PART1
2. GITHUB: GomolemoDizamuhupe/PROG6212_PART_1

Section A 2
1. ERD 3
Section B 3
2. User Profile 4
3. Events 5
4. Categories 6
5. Routes 7
6. Entries 9
7. Results 10
Section C 11
1.CREATING DATABASE AND TABLES 12
2.INSERTING INTO TABLES 12
LINKS 13
REFERENCES 14

Section A

1. ERD



Section B

1. Authentication

Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/auth/register	Registers a new user as either an Organiser or a Participant.	None	{ name, surname, email, password, role }	201 Created — user object (no password). 400 Bad Request — validation error. 409 Conflict — email already registered.
POST	/api/auth/login	Authenticates a user and returns a JWT access token.	None	{ email, password }	200 OK — { token, userId, role }. 401 Unauthorized — invalid credentials.

2. User Profile

Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/users/me	Returns the profile of the currently logged-in user.	Any	None	200 OK — user profile object. 401 Unauthorized — no/invalid token.
PUT	/api/users/me	Updates the logged-in user's own profile details.	Any	{ name, surname, email }	200 OK — updated profile. 400 Bad

					Request — validation error.
GET	/api/users/{userId}	Returns a specific user's public profile.	Organiser	None	200 OK — user object. 404 Not Found — user does not exist.

3. Events

Method	Route	Description	Role Required	Request Body	Expected Response
GET	/api/events	Lists all upcoming events, with optional filters (location, date).	None	None	200 OK — array of events.
GET	/api/events/{eventId}	Returns full details for a single event, including its categories.	None	None	200 OK — event object with nested categories. 404 Not Found.
POST	/api/events	Creates a new event, owned by the logged-in organiser.	Organiser	{ name, description, eventDate, location }	201 Created — event object. 400 Bad Request — validation error.
PUT	/api/events/{eventId}	Updates an existing event owned by the logged-in organiser.	Organiser	{ name, description, eventDate, location }	200 OK — updated event. 403 Forbidden — not the

					owning organiser. 404 Not Found.
DELETE	/api/events/{eventId}	Deletes an event owned by the logged-in organiser.	Organiser	None	204 No Content. 403 Forbidden. 404 Not Found.

4. Categories

Method	**Route**	**Description**	**Role Required**	**Request Body**	**Expected Response**
GET	/api/events/{eventId}/categories	Lists all categories (e.g. 10km Race, Wheelchair Division) for a specific event.	None	None	200 OK — array of categories. 404 Not Found — event does not exist.
GET	/api/categories/{categoryId}	Returns a single category, including its route.	None	None	200 OK — category object with nested route. 404 Not Found.

POST	/api/events/{eventId}/categories	Adds a new category to an event.	Organiser	{ name, distanceKm, maxParticipants }	201 Created — category object. 400 Bad Request. 403 Forbidden — not the owning organiser.
PUT	/api/categories/{categoryId}	Updates a category's details.	Organiser	{ name, distanceKm, maxParticipants }	200 OK — updated category. 403 Forbidden. 404 Not Found.
DELETE	/api/categories/{categoryId}	Removes a category from an event.	Organiser	None	204 No Content. 403 Forbidden. 404 Not Found.

5. Routes

Method	**Route**	**Description**	**Role Required**	**Request Body**	**Expected**

					Response
GET	/api/categories/{categoryId}/route	Returns the route (start/end point, elevation, map) for a category, for race-day prep.	None	None	200 OK — route object. 404 Not Found — category has no route defined.
POST	/api/categories/{categoryId}/route	Creates the route for a category (one route per category).	Organiser	{ startPoint, endPoint, elevationGain, mapUrl }	201 Created — route object. 403 Forbidden — not the owning organiser. 409 Conflict — category already has a route.
PUT	/api/routes/{routeId}	Updates an existing route's details.	Organiser	{ startPoint, endPoint, elevationGain, mapUrl }	200 OK — updated route. 403 Forbidden. 404

					Not Found.

6. Entries

Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/categories/{categoryId}/entries	Enters the logged-in participant into a category (race entry).	Participant	None	201 Created — entry object with status. 404 Not Found — category does not exist. 409 Conflict — already entered, or category full.
GET	/api/users/me/entries	Lists the logged-in participant's own entries across all events.	Participant	None	200 OK — array of entries.
GET	/api/events/{eventId}/entries	Lists all entries for an event (for the	Organiser	None	200 OK — array of entries with

		organiser managing it).			participant details. 403 Forbidden — not the owning organiser.
DELETE	/api/entries/{entryId}	Withdraws an entry (self-service cancellation).	Participant	None	204 No Content. 403 Forbidden — not the owning participant. 404 Not Found.

7. Results

Method	Route	Description	Role Required	Request Body	Expected Response
POST	/api/entries/{entryId}/results	Captures the finishing result for a participant's entry. capturedBy is set to the logged-in organiser.	Organiser	{ finishTime, position }	201 Created — result object. 403 Forbidden — not the owning organiser. 404 Not Found — entry does not exist. 409 Conflict — result

					already captured.
PUT	/api/results/{resultId}	Corrects a previously captured result.	Organiser	{ finishTime, position }	200 OK — updated result. 403 Forbidden. 404 Not Found.
GET	/api/users/me/results	Returns the logged-in participant's full personal result history.	Participant	None	200 OK — array of results with event/category context.
GET	/api/events/{eventId}/results	Returns the full results list for an event (leaderboard).	None	None	200 OK — array of results ordered by position. 404 Not Found.

Section C

1.CREATING DATABASE AND TABLES

```sql
CREATE DATABASE RACEDAY_DB;
USE RACEDAY_DB;

CREATE TABLE Users (
    UserId INT IDENTITY(1,1) PRIMARY KEY,
    Name VARCHAR(100) NOT NULL,
    Surname VARCHAR(100) NOT NULL,
    Email VARCHAR(150) NOT NULL UNIQUE,
    PasswordHash VARCHAR(255) NOT NULL,
    Role VARCHAR(20) NOT NULL
    CONSTRAINT CK_Users_Role
    CHECK (Role IN ('organiser', 'participant'))
);

CREATE TABLE Events (
    EventId INT IDENTITY(1,1) PRIMARY KEY,
    OrganiserId INT NOT NULL,
    Name VARCHAR(150) NOT NULL,
    Description VARCHAR(255) NULL,
    EventDate DATE NOT NULL,
    Location VARCHAR(150) NOT NULL,
    FOREIGN KEY (OrganiserId) REFERENCES Users(UserId)
);
```

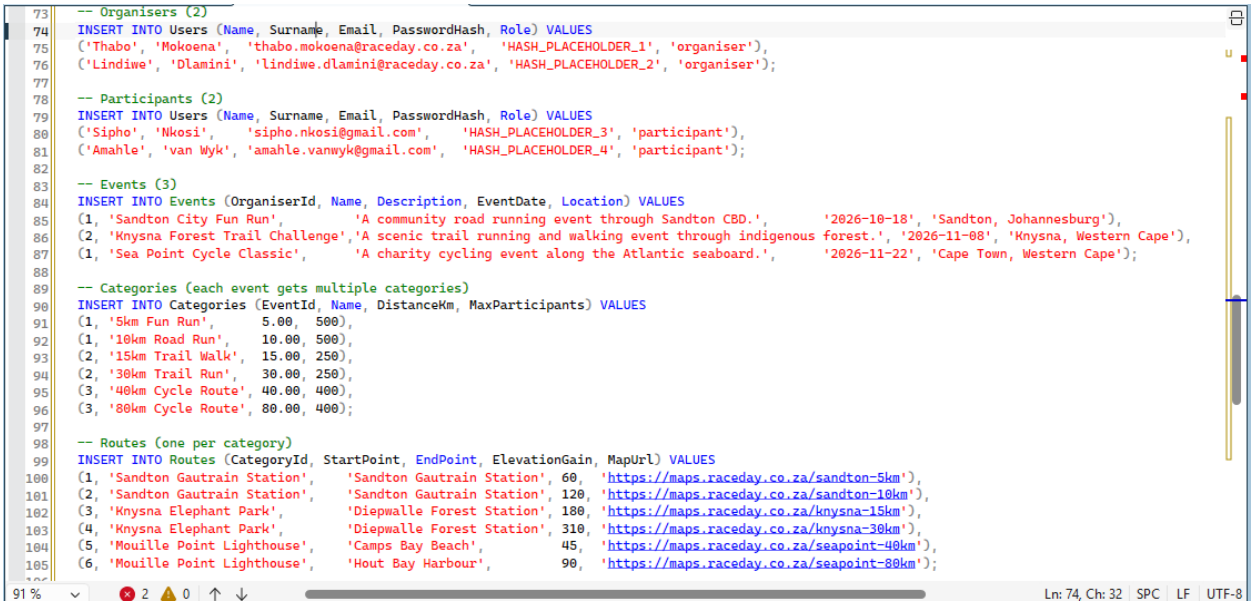
```sql
CREATE TABLE Categories (
    CategoryId INT IDENTITY(1,1) PRIMARY KEY,
    EventId INT NOT NULL,
    Name VARCHAR(50) NOT NULL,
    DistanceKm DECIMAL(5,2) NOT NULL,
    MaxParticipants INT NOT NULL DEFAULT 500,
    CONSTRAINT FK_Categories_Event FOREIGN KEY (EventId)
    REFERENCES Events(EventId)
);

CREATE TABLE Routes (
    RouteId INT IDENTITY(1,1) PRIMARY KEY,
    CategoryId INT NOT NULL,
    StartPoint VARCHAR(150) NOT NULL,
    EndPoint VARCHAR(150) NOT NULL,
    ElevationGain INT NULL,
    MapUrl VARCHAR(255) NULL,
    CONSTRAINT FK_Routes_Category FOREIGN KEY (CategoryId)
    REFERENCES dbo.Categories(CategoryId)
);
```

```sql
CREATE TABLE Entries (
    EntryId INT IDENTITY(1,1) PRIMARY KEY,
    ParticipantId INT NOT NULL,
    CategoryId INT NOT NULL,
    EntryDate DATETIME NOT NULL,
    Status VARCHAR(20) NOT NULL DEFAULT 'Confirmed'
    CONSTRAINT CK_Entries_Status CHECK (Status IN ('Pending', 'Confirmed', 'Cancelled')),
    CONSTRAINT FK_Entries_Participant FOREIGN KEY (ParticipantId)
    REFERENCES Users(UserId),
    CONSTRAINT FK_Entries_Category FOREIGN KEY (CategoryId)
    REFERENCES Categories(CategoryId),
    CONSTRAINT UQ_Entries_Participant_Category UNIQUE (ParticipantId, CategoryId)
);

CREATE TABLE Results (
    ResultId INT IDENTITY(1,1) PRIMARY KEY,
    EntryId INT NOT NULL UNIQUE,
    FinishTime TIME NOT NULL,
    Position INT NULL,
    CapturedBy INT NOT NULL,
    CONSTRAINT FK_Results_Entry FOREIGN KEY (EntryId)
    REFERENCES Entries(EntryId),
    CONSTRAINT FK_Results_CapturedBy FOREIGN KEY (CapturedBy)
    REFERENCES Users(UserId)
);
```

2.INSERTING INTO TABLES



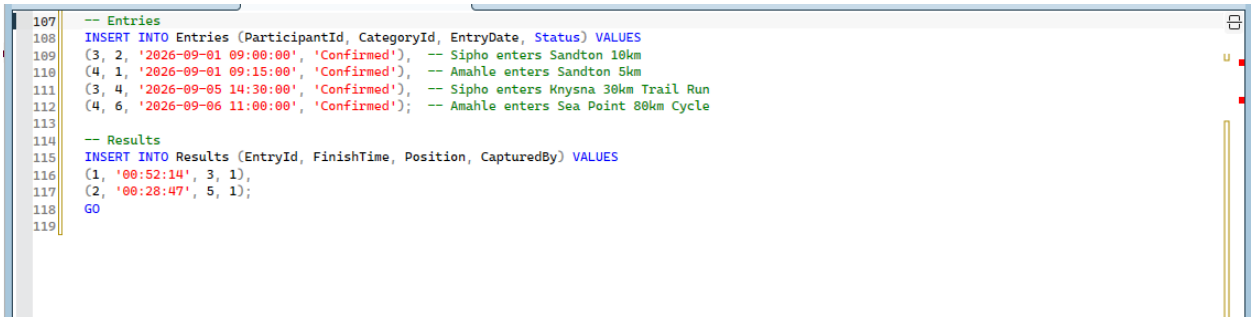
```sql
-- Organisers (2)
INSERT INTO Users (Name, Surname, Email, PasswordHash, Role) VALUES
('Thabo', 'Mokoena',  'thabo.mokoena@raceday.co.za',   'HASH_PLACEHOLDER_1', 'organiser'),
('Lindiwe', 'Dlamini', 'lindiwe.dlamini@raceday.co.za', 'HASH_PLACEHOLDER_2', 'organiser');

-- Participants (2)
INSERT INTO Users (Name, Surname, Email, PasswordHash, Role) VALUES
('Sipho', 'Nkosi',    'sipho.nkosi@gmail.com',    'HASH_PLACEHOLDER_3', 'participant'),
('Amahle', 'van Wyk', 'amahle.vanwyk@gmail.com',  'HASH_PLACEHOLDER_4', 'participant');

-- Events (3)
INSERT INTO Events (OrganiserId, Name, Description, EventDate, Location) VALUES
(1, 'Sandton City Fun Run',        'A community road running event through Sandton CBD.',       '2026-10-18', 'Sandton, Johannesburg'),
(2, 'Knysna Forest Trail Challenge','A scenic trail running and walking event through indigenous forest.', '2026-11-08', 'Knysna, Western Cape'),
(1, 'Sea Point Cycle Classic',     'A charity cycling event along the Atlantic seaboard.',      '2026-11-22', 'Cape Town, Western Cape');

-- Categories (each event gets multiple categories)
INSERT INTO Categories (EventId, Name, DistanceKm, MaxParticipants) VALUES
(1, '5km Fun Run',       5.00,  500),
(1, '10km Road Run',     10.00, 500),
(2, '15km Trail Walk',   15.00, 250),
(2, '30km Trail Run',    30.00, 250),
(3, '40km Cycle Route', 40.00, 400),
(3, '80km Cycle Route', 80.00, 400);

-- Routes (one per category)
INSERT INTO Routes (CategoryId, StartPoint, EndPoint, ElevationGain, MapUrl) VALUES
(1, 'Sandton Gautrain Station',    'Sandton Gautrain Station', 60,  'https://maps.raceday.co.za/sandton-5km'),
(2, 'Sandton Gautrain Station',    'Sandton Gautrain Station', 120, 'https://maps.raceday.co.za/sandton-10km'),
(3, 'Knysna Elephant Park',        'Diepwalle Forest Station', 180, 'https://maps.raceday.co.za/knysna-15km'),
(4, 'Knysna Elephant Park',        'Diepwalle Forest Station', 310, 'https://maps.raceday.co.za/knysna-30km'),
(5, 'Mouille Point Lighthouse',    'Camps Bay Beach',          45,  'https://maps.raceday.co.za/seapoint-40km'),
(6, 'Mouille Point Lighthouse',    'Hout Bay Harbour',         90,  'https://maps.raceday.co.za/seapoint-80km');
```



```sql
-- Entries
INSERT INTO Entries (ParticipantId, CategoryId, EntryDate, Status) VALUES
(3, 2, '2026-09-01 09:00:00', 'Confirmed'),  -- Sipho enters Sandton 10km
(4, 1, '2026-09-01 09:15:00', 'Confirmed'),  -- Amahle enters Sandton 5km
(3, 4, '2026-09-05 14:30:00', 'Confirmed'),  -- Sipho enters Knysna 30km Trail Run
(4, 6, '2026-09-06 11:00:00', 'Confirmed');  -- Amahle enters Sea Point 80km Cycle

-- Results
INSERT INTO Results (EntryId, FinishTime, Position, CapturedBy) VALUES
(1, '00:52:14', 3, 1),
(2, '00:28:47', 5, 1);
GO
```

LINKS

3. YOUTUBE: PROG6212 PART1
4. GITHUB: GomolemoDizamuhupe/PROG6212_PART_1

REFERENCES

diagrams.net (Draw.io), 2026. *Draw.io: Free online diagram software*. Available at: https://www.drawio.com/ [Accessed 2026].

Microsoft, 2026. *SQL Server Management Studio (SSMS) documentation*. Available at: https://learn.microsoft.com/en-us/sql/ssms/ [Accessed 2026].